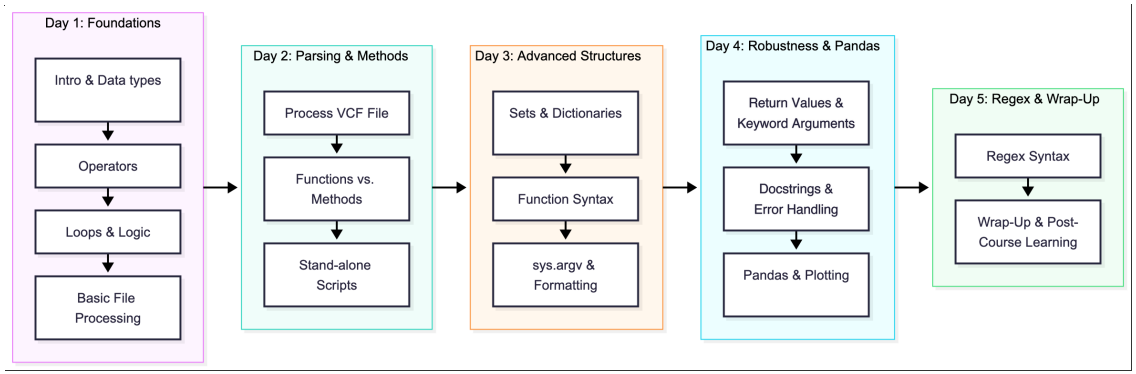
Introduction to



with Application to Bioinformatics

- Day 5

Yesterday's Key Takeaways



- Error handling using `try..except`, which helps you catch problems and prevent your program from crashing.
- Keyword arguments and positional arguments, which give you more control, readability, and flexibility when calling functions.
- Docstrings and comments, which provide clear documentation and make your code more understandable and maintainable.
- Pandas, a powerful and widely used library for working with tabular data.

Day 5

- **Session 1**
    - Quiz: Review of Day 4
    - Lecture: Go through questions
    - Lecture: Introduction to regex
    - Ex1: Find the pattern using regex
- **Session 2**
    - Lecture: Regex in Python
    - Ex2: Regex using Python
    - PyQuiz 5.1
- **Session 3**
    - Lecture: Sum up
    - Presentation about NBIS
    - Ex3: Final exercise
- **Project time**

Quiz: Review Day 4

Go to Canvas, `Modules -> Day 5 -> Review Day 4`

~15 minutes

1. What will be printed when you run the following code?

In [ ]:
```
def gc_count(seq):
    print(seq.count('G') + seq.count('C'))
    return 0

result = gc_count('ATGC')
print(result)
```

2. What is the difference between positional arguments and keyword arguments?

Keyword arguments can be given in any order, while positional arguments depend on the function's order

2. What is the difference between positional arguments and keyword arguments?

Keyword arguments can be given in any order, while positional arguments depend on the function's order

In [ ]:
```
def sum3(a, b, c):
    print(f"a={a}")
    print(f"b={b}")
    print(f"c={c}")
    return a+b+c
sum3(1, c=2, b=3)
```

2. What is the difference between positional arguments and keyword arguments?

Keyword arguments can be given in any order, while positional arguments depend on the function's order

In [ ]:
```
def sum3(a, b, c):
    print(f"a={a}")
    print(f"b={b}")
    print(f"c={c}")
    return a+b+c
sum3(1, c=2, b=3)
```

Arguments can be used in both ways, with or without keyword, if there is no ambiguity

- When used with keyword, they are keyword arguments
- When used without keyword, they are positional arguments

3. What will be the output of the following code snippet?

In [ ]:
```
def add(x=0, y=1, z=0):
    return x + y + z
print(add(1, 2))
print(add(1, y=2, z=3))
print(add())
```

4. Why is it beneficial to use docstrings in functions?

- They provide explanations and details about the function for others reading your code. **Both `"""` and `'''` can be used for docstring**

4. Why is it beneficial to use docstrings in functions?

- They provide explanations and details about the function for others reading your code. **Both `"""` and `'''` can be used for docstring**

In [ ]:
```
def add(x, y, z=0):
    """This function adds up three numbers, with the last one default to
    # Here we secretly increment x by 1
    x += 1
    return x + y + z
help(add)
```

5. How can you see the documentation of a Python library function in the console?

- Use help(library.function)

In [ ]:
```
help(print)
```

6. What will be printed when you run the following code?

In [ ]:
```
values = ['10', 'abc', '20']
total = 0
for v in values:
    try:
        total += int(v)
    except ValueError:
        print('Skipping', v)

print(total)
```

7. The user types 42 when the following code runs. What is the type of x?

In [ ]:
```
x = input('Give a number: ')
print(type(x))
```

8. If you want to filter rows in `df` where `age` is greater than 30, which command would you use?

- df[df['age'] > 30]

8. If you want to filter rows in `df` where `age` is greater than 30, which command would you use?

- df[df['age'] > 30]

In [ ]:
```
import pandas as pd
df = pd.DataFrame({
    'Tree': [1, 1, 3, 4],
    'age': [25, 600, 700, 40],
    'height': [165.4, 175.3, 168.5, 180.6]
})
print(df)
df[df['age'] > 30]
```

9. Which command selects the rows where Tree is 1 and age is greater than 500?

In [ ]:
```
print(df)
df[[df['Tree'] == 1] | [df['age'] > 500]]
```

Regular Expressions

- A regular expression (regex or regexp) is a sequence of characters that defines a search pattern.
- Use case: regular expressions are used in text processing for searching, matching, and manipulating strings

In [ ]:

Examples where regex can play a role

- Find variations in a protein or DNA sequence
    - `"MVR???A"`
    - `"ATG???TAG"`
- American/British spelling, endings and other variants:
    - salpeter, salpetre, saltpeter, nitre, niter or KNO3
    - hemaglobin, heamoglobin, hemaglobins, heamoglobin's
    - catalyze, catalyse, catalyzed...
- A pattern in a VCF or GTF file
    - a digit appearing after a tab

Regex is not unique for Python and it is supported by

- most programming languages,
- text editors
- command line tools (e.g. `grep`, `sed`)
- search engines

Regex is not unique for Python and it is supported by

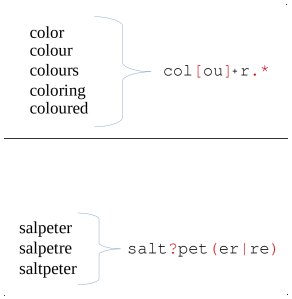
- most programming languages,
- text editors
- command line tools (e.g. `grep`, `sed`)
- search engines

In [ ]:
```
!grep "furniture.*sell" ../downloads/blocket_listings.txt
```

A playground for regex with detailed explanations of your regex

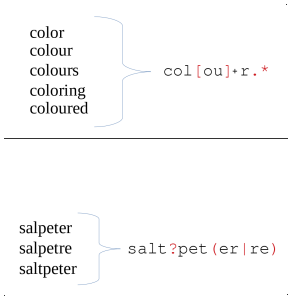
https://regex101.com/

Defining a search pattern



- `.` matches any character (once)
- `?` repeat previous pattern 0 or 1 times
- `*` repeat previous pattern 0 or more times
- `+` repeat previous pattern 1 or more times

Defining a search pattern



- `.` matches any character (once)
- `?` repeat previous pattern 0 or 1 times
- `*` repeat previous pattern 0 or more times
- `+` repeat previous pattern 1 or more times

In [ ]:
```
colou?r.* colou  [u]   colu
```

More common operations - classes of characters

- `\w` matches any letter or number, and the underscore
- `\d` matches any digit
- `\D` matches any non-digit
- `\s` matches any whitespace (spaces, tabs, ...)
- `\S` matches any non-whitespace

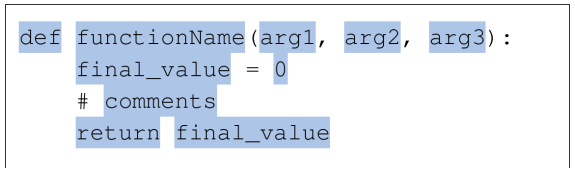
In [ ]:

More common operations - classes of characters

- `\w` matches any letter or number, and the underscore

`\w+`

```
def functionName(arg1, arg2, arg3):
    final_value = 0
    # comments
    return final_value
```



More common operations - classes of characters

- `\d` matches any digit

In [ ]:

`\d+`

```
def functionName(arg1, arg2, arg3):
    final_value = 0
    # comments
    return final_value
```

More common operations - classes of characters

- `\s` matches any whitespace (spaces, tabs, ...)

`\s+`

```
def functionName(arg1, arg2, arg3):
    final_value = 0
    # comments
    return final_value
```

More common operations - classes of characters

- `[abc]` matches a single character defined in this set {a, b, c}
- `[^abc]` matches a single character that is **not** a, b or c
- `[a-z]` matches all letters between `a` and `z` (the english alphabet).
- `[a-z]+` matches any (lowercased) english word.

More common operations - classes of characters

- `[abc]` matches a single character defined in this set {a, b, c}
- `[^abc]` matches a single character that is **not** a, b or c
- `[a-z]` matches all letters between `a` and `z` (the english alphabet).
- `[a-z]+` matches any (lowercased) english word.

```
plaintext
Three cats and 11 dogs run fast across grass fields.
```

In [ ]:
```
# matches any single character not a letter and not a space.
[^a-zA-Z ]
```

Some regular expressions are equivalent; choose the one that you prefer

Shorthand	Equivalent Range	Description
`\d`	`[0-9]`	Matches any digit (0-9).
`\D`	`[^0-9]`	Matches any non-digit character.
`\w`	`[a-zA-Z0-9_]`	Matches any word character (letters, numbers, and the underscore).
`\W`	`[^\w]`	Matches any non-word character (e.g., symbols, spaces).
`\s`	`[ \t\n\r\f\v]`	Matches any whitespace character (space, tab, newline, etc.).
`\S`	`[^\s]`	Matches any non-whitespace character.

Example - finding patterns in a VCF file

```
1   920760 rs80259304 T   C   .   PASS
AA=T;AC=18;AN=120;DP=190;GP=1:930897;BN=131   GT:DP:CB   0/1:1:SM
0/0:4:SM...
```

Find one record with at least one sample, that is, having genotype fields such as `0/0:1:SM` `0/0:4:SM`

Example - finding patterns in a VCF file

```
1   920760 rs80259304 T   C   .   PASS
AA=T;AC=18;AN=120;DP=190;GP=1:930897;BN=131   GT:DP:CB   0/1:1:SM
0/0:4:SM...
```

Find one record with at least one sample, that is, having genotype fields such as `0/0:1:SM` `0/0:4:SM`

- `"[01]/[01]"` (or `"\d/\d"`)
- `\s\d/\d:`

Example - finding patterns in a VCF file

`1  920760 rs80259304 T  C  .  PASS  AA=T;AC=18;AN=120;DP=190;GP=1:930897;BN=131  GT:DP:CB  0/1:1:SM  0/0:4:SM...`

Find one record with at least one sample, that is, having genotype fields such as **`0/0:1:SM`** **`0/0:4:SM`**

- `"[01]/[01]"` (or `"\d/\d"`)
- `\s\d/\d:`

In [ ]:
```
!grep -E "[01]/[01]" ../downloads/genotypes_small.vcf | head -n 1
```

Example - finding patterns in vcf

```
1   920760 rs80259304 T   C   .   PASS
AA=T;AC=18;AN=120;DP=190;GP=1:930897;BN=131   GT:DP:CB   0/1:1:SM
0/0:4/SM...
```

- Find all lines containing more than one homozygous alternate sample (assuming all homozygous alternate are of the form 1/1, which might not be the case in general)

Example - finding patterns in vcf

```
1   920760 rs80259304 T   C   .   PASS
AA=T;AC=18;AN=120;DP=190;GP=1:930897;BN=131   GT:DP:CB   0/1:1:SM
0/0:4/SM...
```

- Find all lines containing more than one homozygous alternate sample (assuming all homozygous alternate are of the form 1/1, which might not be the case in general)

```
... 1/1:...  ... 1/1:...  ...
```

Example - finding patterns in vcf

`1  920760 rs80259304 T  C  .  PASS  AA=T;AC=18;AN=120;DP=190;GP=1:930897;BN=131  GT:DP:CB  0/1:1:SM  0/0:4/SM...`

- Find all lines containing more than one homozygous alternate sample (assuming all homozygous alternate are of the form 1/1, which might not be the case in general)

`... 1/1:...  ... 1/1:...  ...`

- `.*1/1.*1/1.*`
- `.*\s1/1:.*\s1/1:.*`

In [ ]:
```
sed -E ".*\s1/1:.*\s1/1:.*" ../downloads/genotypes_small.vcf  | wc -l
```

Example from the project (with GTF file):

how many transcripts can the CFTR gene (ENSG00000001626) generate?

In [ ]:
```
!grep -E "transcript.*ENSG00000001626" ../assignment/data/Homo_sapiens.G
```

Cheat sheet

- `.` matches any character (once)
- `?` repeat previous pattern 0 or 1 times
- `*` repeat previous pattern 0 or more times
- `+` repeat previous pattern 1 or more times
- `\w` matches any letter or number, and the underscore
- `\d` matches any digit
- `\D` matches any non-digit
- `\s` matches any whitespace (spaces, tabs, ...)
- `\S` matches any non-whitespace
- `[abc]` matches a single character defined in this set {a, b, c}
- `[^abc]` matches a single character that is **not** a, b or c
- `[a-z]` matches any (lowercased) letter from the english alphabet
- `.*` matches anything, but **not cross multiple lines**.

Day 5, Exercise 1 (~30 min)

Practicing regular expressions

- Canvas -> Modules -> Day 5 -> Exercise 1 - day 5

Start the exercise by running

---

python retester.py

---

in the `downloads` folder in a terminal

Take a break after the exercise (~10 min)

Session 2

- How to use regex in Python
- Ex2: Regex using Python
- PyQuiz 5.1

Regular expressions in Python

Regular expressions in Python

In [ ]:

```
import re
pattern = r"col[ou]+r.*"
text = "The colour of the wall is very vibrant,\
but the color of the sky is even more spectacular."
# Try to find a hit
result = re.search(pattern, text)
if result:
    print(result.group())
    # print(result.span())
    # print(result.span()[0])
    # print(result.start())
else:
    print("No match found")
```

Regular expressions in Python

In [ ]:
```python
import re
pattern = r"col[ou]+r.*"
text = "The colour of the wall is very vibrant,\
but the color of the sky is even more spectacular."
# Try to find a hit
result = re.search(pattern, text)
if result:
    print(result.group())
    # print(result.span())
    # print(result.span()[0])
    # print(result.start())
else:
    print("No match found")
```

`result.group()` : Return the string matched by the expression

`result.start()` : Return the starting position of the match

`result.end()` : Return the ending position of the match

`result.span()` : Return both (start, end)

Raw string

```
pattern = r"col[ou]+r.*"
```

- A raw string is a Python string literal prefixed with the letter `r`.
- Highly recommended to use raw string `(r"...")` for regular expression patterns in Python.
- Raw strings simplify pattern writing by preventing Python from interpreting backslashes `\` as special escape sequences.

Raw string

```
pattern = r"col[ou]+r.*"
```

- A raw string is a Python string literal prefixed with the letter `r`.
- Highly recommended to use raw string `(r"...")` for regular expression patterns in Python.
- Raw strings simplify pattern writing by preventing Python from interpreting backslashes `\` as special escape sequences.

In [ ]:
```
text = "The colour of the wall is very vibrant, but the color of the sky
pattern = "\bthe\b" # \b matches the word boundary, try with and without
                    # without 'r', \b is interpreted as ASCII backspace
result = re.search(pattern, text)
if result:
    print(result.group())
else:
    print("No match found")
```

In [ ]:
```

```

In [ ]:

```
# How if we only want to search for the words "colour", "color"?
pattern = r"col[ou]+r.*"
text = "The colour of the wall is very vibrant, but the color of the sky
# pattern = r"col[ou]+r\b"
result = re.search(pattern, text)
if result:
    print(f"Found text: '{result.group()}'")
    print(result.span())
else:
    print("No match found")
```

How to find all occurences of "color" variations?

How to find all occurences of "color" variations?

```
re.findall
```

How to find all occurences of "color" variations?

```
re.findall
```

In [ ]:
```
text = "The colour of the wall is very vibrant, but the color of the sky
pattern = r"col[ou]+r\b"
result = re.findall(pattern, text)
print(result)
```

How to find all occurences of "color" variations with their positions in the text?

How to find all occurences of "color" variations with their positions in the text?

```
re.finditer
```

How to find all occurences of "color" variations with their positions in the text?

```
re.finditer
```

In [ ]:

```
text = "The colour of the wall is very vibrant, but the color of the sky
pattern = r"col[ou]+r\b"

for result in re.finditer(pattern, text):
    print(f"Find word: {result.group()} at {result.span()}")
```

re.compile

re.compile

In [ ]:
```
text = "The colour of the wall is very vibrant, but the color of the sky
pattern = r"col[ou]+r\b"
p_find_colour = re.compile(pattern)
#print(type(p_find_colour))
p_find_colour.search(text)
# re.search(pattern, text)
# p_find_colour.findall("colur colour color")
```

re.compile

In [ ]:
```
text = "The colour of the wall is very vibrant, but the color of the sky
pattern = r"col[ou]+r\b"
p_find_colour = re.compile(pattern)
#print(type(p_find_colour))
p_find_colour.search(text)
# re.search(pattern, text)
# p_find_colour.findall("colur colour color")
```

In [ ]:
```
p_find_colour.findall(text)
```

re.compile

In [ ]:
```
text = "The colour of the wall is very vibrant, but the color of the sky
pattern = r"col[ou]+r\b"
p_find_colour = re.compile(pattern)
#print(type(p_find_colour))
p_find_colour.search(text)
# re.search(pattern, text)
# p_find_colour.findall("colur colour color")
```

In [ ]:
```
p_find_colour.findall(text)
```

In [ ]:
```
for result in p_find_colour.finditer(text):
    print(f"Find colour at the position {result.span()}, the word is '{r
```

Benefits of using `re.compile`

- Improved performance - saves time by compiling the regex only once, especially when used in a loop
- Reusability - the compiled regex can be used multiple times
- Early error detection - syntax error of the pattern can be detected at the compiling stage instead of mixed with the other `re` functions

Case insensitiveness

In [ ]:
```
# Remember, [a-z]+ matches any lower case English word
pattern = r'[a-z]+'
result = re.search(pattern, '123 ATGAAA 456')
if result:
    print(result)
else:
    print("No match found")
```

Case insensitiveness

In [ ]:
```
# Remember, [a-z]+ matches any lower case English word
pattern = r'[a-z]+'
result = re.search(pattern, '123 ATGAAA 456')
if result:
    print(result)
else:
    print("No match found")
```

In [ ]:
```
pattern = r'[a-z]+'

result = re.search(pattern, '123 ATGAAA 456', flags=re.IGNORECASE)
if result:
    print(f"Found hit {result.group()} at the position {result.span()}")
else:
    print("No match found")
```

How to find a full stop?

In [ ]:
```
text = "The first full stop is here: ."

result = re.search(r".", text)
print(f"Found {result.group()} at position {result.start()}")
```

How to find a full stop?

In [ ]:
```
text = "The first full stop is here: ."

result = re.search(r".", text)
print(f"Found {result.group()} at position {result.start()}")
```

In [ ]:
```
# Use the escape character \
result = re.search(r"\.", text)
print(f"Found {result.group()} at position {result.start()}")
```

More operations

- `\` escaping a character, use the character as is
- `^` beginning of the string r"^abc"
- `$` end of string
- `|` boolean `or`

Matching beginning and end of line

```
^hello$
```

Matching beginning and end of line

`^hello$`

In [ ]:
```
text = "hello   \nhello\n    hello"
# print(text)
result = re.search(r'^hello$', text, flags=re.MULTILINE)
if result:
    print(f"Find word '{result.group()}' at position {result.span()}")
else:
    print("No match found")
```

Substitution

Substitution

In [ ]:
```
text = "Do it   becuase   I say so,     not becuase you want!"
# Spell the word because correctly
re.sub(r'becuase', 'because', text)
print(text)
```

Substitution

In [ ]:
```
text = "Do it   becuase   I say so,     not becuase you want!"
# Spell the word because correctly
re.sub(r'becuase', 'because', text)
print(text)
```

In [ ]:
```
text = re.sub(r'becuase', 'because', text)
print(text)
```

Substitution

In [ ]:
```
text = "Do it   becuase   I say so,     not becuase you want!"
# Spell the word because correctly
re.sub(r'becuase', 'because', text)
print(text)
```

In [ ]:
```
text = re.sub(r'becuase', 'because', text)
print(text)
```

In [ ]:
```
# Remove additional spaces
re.sub(r'\s+', ' ', text)
```

Example from Day 2 when lstrip() does not work as expected

In [ ]:
```
line = "AB:A:B should not be removed"
line.lstrip("AB:") # The chars argument is not a prefix
```

In [ ]:
```
line.removeprefix("AB:")
```

Example from Day 2 when lstrip() does not work as expected

In [ ]:
```
line = "AB:A:B should not be removed"
line.lstrip("AB:") # The chars argument is not a prefix
```

In [ ]:
```
line.removeprefix("AB:")
```

In [ ]:
```
newline = re.sub(r"^AB:", "", line)
print(newline)
```

Overview

- Construct regular expressions

```
p = re.compile()
```

- Searching

```
p.search(text)
re.search(pattern, text)
```

- Substitution

```
p.sub(replacement, text)
```

Typical code structure for text matching:

```
pattern = re.compile(r"regex")
result = pattern.search('string goes here')

if result:
    print('Match found: ', result.group())
else:
    print('No match')
```

Summary

- A powerful tool to search and modify text
- There is much more to read in the docs
- Note: regex comes in different flavours. If you use it outside Python, there might be small variations in the syntax.

Day 5, Exercise 2 (~30 min)

Use regular expressions with Python

- Canvas -> Modules -> Day 5 -> Exercise 2 - day 5
- Do bonus exercise only when you are super interested in RegEx

Take a break after the exercise (~10 min)

PyQuiz 5.1 (~10 min)

Lunch

Sum up!

Processing files - looping through the lines

```
with open('myfile.txt', 'r') as fh
    for line in fh:
        do_stuff(line)
```

In [ ]:

```
with open('myfile.txt', 'r', encoding='utf-8') as fh
    for line in fh:
        do_stuff(line)
```

Store values

```
iterations = 0
information = []

with open('myfile.txt', 'r') as fh:
    for line in fh:
        iterations += 1
        information += do_stuff(line)
```

Values

- Base types:

```
str     "hello"
int     5
float   5.2
bool    True
```

- Collections:

```
list  ["a", "b", "c"]
dict  {"a": "alligator", "b": "bear", "c": "cat"}
tuple ("this", "that")
set   {"drama", "sci-fi"}
```

Assign values

```
iterations = 0
score      = 5.2
# variable = literal
```

Compare and membership

```
+, -, *,...   # mathematical
and, or, not  # logical
==, !=        # (in)equality
<, >, <=, >=  # comparison
in            # membership
```

Showing the result: use `f-string`

In [ ]:
```
# Use f-string for print out result
value = 4
nextvalue = 1
nextvalue += value
print(f"nextvalue: {nextvalue}, value: {value}")
```

Be aware of the order of logical operators

In [ ]:

```
x = 5
y = 7
z = 0

x > 4 or y == 7  and z > 1
```

Be aware of the order of logical operators

In [ ]:
```
x = 5
y = 7
z = 0

x > 4 or y == 7  and z > 1
```

In [ ]:
```
(x > 4 or y == 7) and z > 1
```

Strings

Works like a list of characters

In [ ]:
```
mystr = "one"
```

In [ ]:
```
mystr += " two" # string concatnation
mystr
```

In [ ]:
```
len(mystr) # get the length
```

In [ ]:
```
"one" in mystr # in operator for substring matching
# mystr.find("one") # return index if find a hit
# mystr.index("one") # Identical to .find(), but handles failure differe
```

String is immutable

In [ ]:
```
mystr = "one"
mystr[1] = "W"
```

String is immutable

In [ ]:
```
mystr = "one"
mystr[1] = "W"
```

In [ ]:
```
mystr = "one"
print(mystr)
mystr = "two"
print(mystr)
```

String is immutable

In [ ]:
```
mystr = "one"
mystr[1] = "W"
```

In [ ]:
```
mystr = "one"
print(mystr)
mystr = "two"
print(mystr)
```

In [ ]:
```
mystr = "one"
print(f"mystr = {mystr}, address = {id(mystr)}")
mystr = "two"
print(f"mystr = {mystr}, address = {id(mystr)}")
```

String manipulation

```
s.strip()  # remove trailing white spaces

s.split()  # split the line into columns

s.upper(), s.lower()  # change the case
```

Regular expressions help you find and replace strings

```
p = re.compile(r'A.A.A')
p.search(dna_seq)

p = re.compile(r'T')
rna_seq = p.sub('U', dna_seq)
```

Regular expressions help you find and replace strings

```
p = re.compile(r'A.A.A')
p.search(dna_seq)

p = re.compile(r'T')
rna_seq = p.sub('U', dna_seq)
```

In [ ]:

```
import re
# the greedy star (*)! Match as long as possible
re.search(r'p.*\sp', 'a python programmer writes python code').group()
```

Regular expressions help you find and replace strings

```
p = re.compile(r'A.A.A')
p.search(dna_seq)

p = re.compile(r'T')
rna_seq = p.sub('U', dna_seq)
```

In [ ]:
```
import re
# the greedy star (*)! Match as long as possible
re.search(r'p.*\sp', 'a python programmer writes python code').group()
```

In [ ]:
```
# To match the shortest possible string, use non-greedy quantifier *?
re.search(r'p.*?\sp', 'a python programmer writes python code').group()
```

Collections

Can contain strings, integer, booleans...

- Most collections are mutable (except **tuple**): you can *add*, *remove*, *change* values
- Lists:

```
mylist.append('value')
```

- Dicts:

```
mydict['key'] = 'value'
```

- Sets:

```
myset.add('value')
```

Collections

- Test for membership:

```
value in myobj
```

- Check size:

```
len(myobj)
```

Lists

- Ordered!

```
mylist = [1, 8, 1, 2, 4]

mylist.sort()
mylist.reverse()
mylist[2]
mylist[-1]
mylist[2:6]
```

In [ ]:
```
mylist = [1, 8, 1, 2, 4]
mylist
```

In [ ]:
```
mylist.sort()
print(mylist)
```

In [ ]:
```
mylist.reverse()
print(mylist)
```

In [ ]:
```
mylist[2]
```

In [ ]:
```
mylist[-1]
```

In [ ]:
```
mylist[2:]
```

Dictionaries

- List of key value pairs

```
mydict = {"a": "alligator", "b": "bear", "c": "cat"}
counter = {"cats": 55, "dogs": 8}

mydict["a"]
mydict.keys()
mydict.values()
mydict.items() # get both keys and values
```

In [ ]:

```
counter = {'cats': 0, 'others': 0}
# counter = {}
for animal in ['zebra', 'cat', 'dog', 'cat']:
    if animal == 'cat':
        counter['cats'] += 1
    else:
        counter['others'] += 1

print(counter)
```

Sets

- Bag of values
    - No order
    - No duplicates
    - Fast membership checks
    - Logical set operations (union, difference, intersection...)

```
myset = {"drama", "sci-fi"}

myset.add("comedy")

myset.remove("drama")
```

In [ ]:

```
set1 = set(["1", "2", "3", "4", "5"])
set1
```

In [ ]:
```
set1 = set(["1", "2", "3", "4", "5"])
set1
```

In [ ]:
```
set1.add("1")
set1
```

In [ ]:
```
set1 = set(["1", "2", "3", "4", "5"])
set1
```

In [ ]:
```
set1.add("1")
set1
```

In [ ]:
```
set2 = set(["3", "6"])
set1.intersection(set2)
```

In [ ]:
```
set1 = set(["1", "2", "3", "4", "5"])
set1
```

In [ ]:
```
set1.add("1")
set1
```

In [ ]:
```
set2 = set(["3", "6"])
set1.intersection(set2)
```

In [ ]:
```
set1.union(set2)
```

In [ ]:
```
set1 = set(["1", "2", "3", "4", "5"])
set1
```

In [ ]:
```
set1.add("1")
set1
```

In [ ]:
```
set2 = set(["3", "6"])
set1.intersection(set2)
```

In [ ]:
```
set1.union(set2)
```

In [ ]:
```
set1.difference(set2)
```

Tuples

- A group (usually two) of values that belong together

```
tup = (max_length, sequence)
```

- An ordered sequence (like lists)

```
length = tup[0]  # get content at index 0
```

- Immutable

Tuples

- A group (usually two) of values that belong together

```
tup = (max_length, sequence)
```

- An ordered sequence (like lists)

```
length = tup[0]  # get content at index 0
```

- Immutable

In [ ]:
```
tup = (2, 'xy')
tup[0]
# type(tuple((list(tup)))) # tuple and list can be converted
```

In [ ]:
```
tup[0] = 2
```

Tuples in functions

```
def find_longest_seq(file):
    # some code here...
    return length, sequence
```

In [ ]:

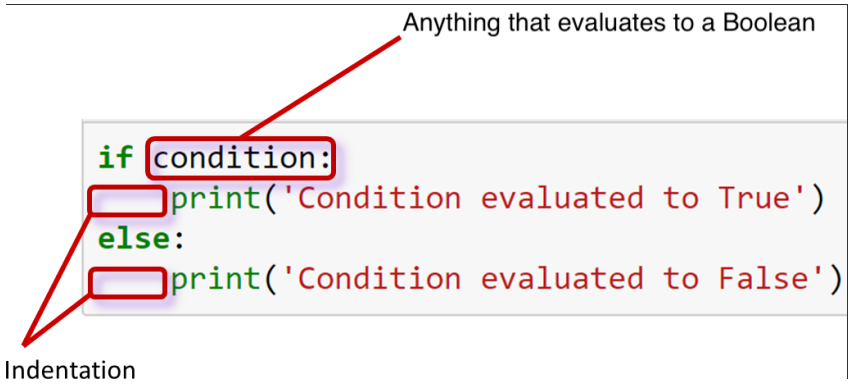
```
def find_longest_seq(file):
    length = 10
    sequence = "AAAAATTTTT"
    return length, sequence

answer = find_longest_seq("dummyfile") # return as a tuple
print(type(answer))
# _, sequence = find_longest_seq("dummyfile") # return as two variables
# print(sequence)
```

Deciding what to do with if else statement

In [ ]:
```
count = 8
if count > 10:
    print('big')
elif count > 5:
    print('medium')
else:
    print('small')
```

Deciding what to do - if statement



`if x:` is equvalent to `if bool(x):`

In [ ]:
```
if [1,4,6]:
    print("yes")
```

Program flow - for loops

In [ ]:
```
information = []
with open('myfile.txt', 'r') as fh
    for line in fh:
        if is_comment(line):
            use_comment(line)
        else:
            information.append(read_data(line)) # read_data return a lis
```

Program flow - while loops

In [ ]:
```
information = []
with open('myfile.txt', 'r') as fh:
    # Read the first line
    line = fh.readline()

    # Continue to read lines until an empty string is returned
    while line:
        information.append(read_data(line)) # read_data return a list
        line = file.readline()  # Read the next line
```

Different types of loops

`For` loop

is a control flow statement that performs operations over a known amount of steps.

`While` loop

is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition.

Which one to use?

`For` loops - standard for iterations over lists and other iterable objects

`While` loops - more flexible and can iterate an unspecified number of times

In [ ]:

```
# For loop example
# You know the number of iterations before hand
user_input = "thank god it's friday"
for letter in user_input:
    print(letter.upper())
```

In [ ]:
```python
# For loop example
# You know the number of iterations before hand
user_input = "thank god it's friday"
for letter in user_input:
    print(letter.upper())
```

In [ ]:
```python
# While loop example
# The number of iterations is unknown before hand
key = ""
max_try = 3
cnt = 0
TARGET_KEY = "python2026"
while key != TARGET_KEY:
    key = input("Enter your key: ")
    cnt += 1
    if cnt > max_try:
        print("WARNING!! Too many failures!")
        break
    if key == TARGET_KEY:
        print("Correct key!")
```

Controlling loops

- `break` - stop the loop
- `continue` - go on to the next iteration

In [ ]:

```
# example for break
user_input = "thank god it's friday"
for letter in user_input:
    if letter == 'd':
        break
    print(letter.upper())
```

In [ ]:
```python
# example for break
user_input = "thank god it's friday"
for letter in user_input:
    if letter == 'd':
        break
    print(letter.upper())
```

In [ ]:
```python
# example for continue
user_input = "thank god it's friday"
for letter in user_input:
    if letter == ' ' or letter == '\'':  # Skip spaces and apostrophes
        continue
    print(letter.upper())
```

Watch out!

In [ ]:
```
# DON'T RUN THIS
i = 0
-while i < 10:
    print(user_input[i])
```

Watch out!

In [ ]:
```
# DON'T RUN THIS
i = 0
-while i < 10:
    print(user_input[i])
```

While loops may be infinite!

File Input/Output

- In: Read from files

```
with open(filename, 'r') as fh:
    for line in fh:
        do_stuff(line)
```

  Read information from command line: `sys.argv[1:]`

- Out: Write to files:

```
with open(filename, 'w') as fh:
    fh.write(text)
```

- Printing:

```
print('my_information')
```

Input/Output

Open files should be closed:

```
      fh.close()
```

or use the `with` clause

```
    with open(filename, "r") as fh:
        do_something
```

Code structure

- Functions (unit)
- Modules (the file)
- Packages (the folder)
- Libraries (the large collection)

Functions

- A named piece of code that performs a certain task.

```
def functionName(arg1, arg2, arg3):

    finalValue = 0

    # Here is some code where you can do
    # calculations etc, on arg1, arg2, arg3
    # and update finalValue

    return FinalValue
```

- Is given a number of input arguments
    - to be used within the function body
- Returns a result (maybe `None`)

Functions - arguments

```
def prettyprinter(name, value, delim=":", end=None):
    out = "The " + name + " is " + delim + " " + value
    if end:
        out += end
    return out
```

- used to set default values (often `None`)
- can be skipped in function calls, then the arguments are positional
- improve readability when keys are used

Using your code

Any longer pieces of code that have been used and will be re-used should be saved

- Save it as a file `mycode.py`
- To run it: `python3 mycode.py` or `python mycode.py`
- Import it: `import mycode`

Documentation and comments

```
""" This is a doc-string explaining what the purpose of this function/
module is """
```

```
# This is a comment that helps understanding the code
```

Documentation and comments

```
""" This is a doc-string explaining what the purpose of this function/
module is """
```

```
# This is a comment that helps understanding the code
```

- Comments *will* help you
- Undocumented code rarely gets used
- Try to keep your code readable: use informative variable and function names

Why programming?

Endless possibilities!

- reverse complement DNA
- custom filtering of VCF files
- plotting of results
- dealing with excel files

Why programming?

- Computers are fast
- Computers don't get bored
- Computers don't get sloppy

Why programming?

- Computers are fast
- Computers don't get bored
- Computers don't get sloppy

- Create reproducable results
    - for you and for others to use
- Extract large amount of information

Type hinting: popular for modern Python coding

In [ ]:

```python
age: int = 30
name: str = "Bob"

def show_new_age(age: int, name: str) -> int:
    # 'age' is expected to be an integer
    # 'name' is expected to be a string
    # The function is expected to return an integer
    new_age: int = age + 10
    print(f"New age of {name} is {new_age}")
    return new_age

show_new_age(age, name)
```

Type hinting: popular for modern Python coding

In [ ]:
```python
age: int = 30
name: str = "Bob"

def show_new_age(age: int, name: str) -> int:
    # 'age' is expected to be an integer
    # 'name' is expected to be a string
    # The function is expected to return an integer
    new_age: int = age + 10
    print(f"New age of {name} is {new_age}")
    return new_age

show_new_age(age, name)
```

- Type hinting improve the documentation and readability
- With proper tools and IDE, type hinting helps early detection of errors and debugging

In [ ]:

```
age: int = 30.0
name: str = "Bob"
print(type(age))
```

In [ ]:
```
age: int = 30.0
name: str = "Bob"
print(type(age))
```

Type hinting does not change how Python interprets your code

Why learning programming

- AI agents are already very powerful and keep evolving rapidly
- AI still makes mistakes
- In research, the answer or expected result format is usually unknown beforehand

Why learning programming

- AI agents are already very powerful and keep evolving rapidly
- AI still makes mistakes
- In research, the answer or expected result format is usually unknown beforehand

- You know the basics - don't be afraid to try, it's the only way to learn
- You will get faster

Final advice

- Getting help
    - Ask colleauges
    - Try speak out your problem
    - Search the web
    - Ask AI and use AI (e.g. Claude, Gemini, ChatGPT, Muse), but with caution.
    - Don't spend too much time on memorizing function and library names etc unless it's your hobby
    - NBIS drop-ins

Now you know Python!



Well done!

You will get a certificate after the course

NBIS presentation

Google Form for anonymous evaluation

Just a small exercise to finish the day and have fun!

Canvas -> Module -> Day 5 -> Exercise 3 - day 5

Exercise 3

In [ ]:
```
!cp ../downloads/checker.py .
# !copy ../downloads/checker.py . # on Windows probabaly
```

In [ ]:
```
import checker
```

In [ ]:
```
checker.check()
```

In [ ]:
```
name = "Shu"
checker.check_name(name)
```

In [ ]:
```

```